LeetCode Practice

2 Hard: String

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

2
HARD

Patterns (in order)

1. String (2)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

2 questions | tap a title to jump to it

Hard questions (2) Go >

String (2) Go >

H 1. Shortest Palindrome LC 214

H 2. Longest Happy Prefix LC 1392

Hard questions (2)

HARD | TOPIC 1 OF 1

String

2 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Shortest Palindrome LC 214
2. Longest Happy Prefix LC 1392

HARD | String | LeetCode #214 | 1 of 2 | 1/2

Shortest Palindrome

QUESTION

You are given a string s. You can convert s to a palindrome by adding characters in front of it.

Return the shortest palindrome you can find by performing this transformation.

Example 1	
Input	`s = "aacecaaa"`
Output	`"aaacecaaa"`

Example 2	
Input	`s = "abcd"`
Output	`"dcbabcd"`

CONSTRAINTS

- 0 <= s.length <= $5 * 10^4$
- s consists of lowercase English letters only.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find shortest palindrome by prepending chars to s. Right?"

"A few quick questions: see constraints."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for L in range(len(s),_1_-_1,_-1):
    if s[0..L] is a palindrome:
        return reverse(s[L..]) + s
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each palindrome check is O(n) and there can be n of them. The longest palindromic prefix of s + '#' + reverse(s) via the KMP failure function is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"KMP failure function on reversed suffix — O(n). Walk-through:"

We want the shortest prefix of s that, when reversed, can be appended to make a palindrome.

Key insight: build combined string = s + '#' + reversed(s).

Run KMP failure function on this combined string.

The last value in the failure table = length of the longest prefix of s that is also

a suffix of reversed(s) = longest palindromic prefix of s.

The answer = reverse(s[k:]) + s, where k = length of that palindromic prefix.

WHY '#' separator: prevents the KMP match from crossing the boundary between s and rev(s).

Example: s='aacecaaa' -> palindromic prefix length=7 -> prepend rev('a') = 'aaacecaaa' ok

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _214_ShortestPalindrome:
    def shortestPalindrome(self, s):
        rev = s[::-1]
        combined = s + '#' + rev
        n = len(combined)
        fail = [0] * n
        j = 0
        for i in range(1, n):
            while j > 0 and combined[i] != combined[j]:
                j = fail[j-1]
            if combined[i] == combined[j]:
                j += 1
            fail[i] = j
        return rev[:len(s)-fail[-1]] + s
```

PHASE 5 TEST & DEBUG

"Let me trace shortestPalindrome on s='aacecaaa'."

Goal: find longest palindrome prefix, add reverse of remaining suffix to front

Build KMP failure fn on t = s + '#' + reverse(s) = 'aacecaaa#aaacecaa'

KMP failure array last value = 7 -> palindrome prefix has length 7 = 'aacecaa'

Non-palindrome suffix = s[7:] = 'a'; add reverse('a')='a' to front

Result: 'a' + 'aacecaaa' = 'aaacecaaa' ok (check: palindrome ok)

WHY KMP: find the longest prefix of s that is also a suffix of reverse(s) = palindrome prefix.

The '#' separator prevents cross-string matching.

Edge: s='' -> ''; s='a' -> 'a' (already palindrome).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) KMP failure function finds longest palindromic prefix."	**O(n)** output or sliding window; O(1) extra if output not counted

Follow-up: LC 5 (Longest Palindromic Substring) uses a similar suffix/prefix palindrome idea — can you apply Manacher's to do it in O(n)?

IN YOUR APP | Expose | Editorial | Community

HARD | String | LeetCode #1392 | 2 of 2 | 2/2

Longest Happy Prefix

QUESTION

A string is called a happy prefix if is a non-empty prefix which is also a suffix (excluding itself).

Given a string s, return the longest happy prefix of s. Return an empty string "" if no such prefix exists.

Example 1

Input	`s = "level"`
Output	`"l"`
Explanation	*s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".*

Example 2

Input	`s = "ababab"`
Output	`"abab"`
Explanation	*"abab" is the largest prefix which is also suffix. They can overlap in the original string.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s contains only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find longest prefix which is also a suffix (KMP failure function). Right?"

"A few quick questions: see constraints."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for l in ls:
    # from n - 1 down to 1
    compare the first L letters with the last L
return the first that match
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Compares prefixes and suffixes character by character for every length. The KMP failure function gives the longest proper prefix that is also a suffix in O(n); a rolling hash also works.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"KMP failure function on the string itself — O(n). Walk-through:"

The longest happy prefix = the longest proper prefix that is also a suffix.

This is exactly what KMP's failure (partial match) table computes.

`failure[i] = length of longest proper prefix of s[:i+1] that is also a suffix.`

Build failure array:

j=0 (length of current matching prefix). i starts at 1.

If s[i]==s[j]: failure[i]=j+1; i++; j++.

Else if j>0: j=failure[j-1] (fall back, don't reset i).

Else: failure[i]=0; i++.

Answer = s[:failure[-1]] (prefix of length = last failure value).

WHY KMP: the failure function is designed to find exactly prefix=suffix overlaps.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1392_LongestHappyPrefix:
    def longestPrefix(self, s):
        n = len(s)
        fail = [0] * n
        j = 0
        for i in range(1, n):
            while j > 0 and s[i] != s[j]:
                j = fail[j-1]
            if s[i] == s[j]:
                j += 1
            fail[i] = j
        return s[:fail[-1]]
```

PHASE 5 TEST & DEBUG

"Let me trace longestPrefix on s='level'."

KMP failure function: for each position find longest proper prefix=suffix

s='level': f=[0,0,0,0,0]? Let's compute:

f[0]=0 (base). f[1]: 'le'->'e'!='l'->0. f[2]: 'lev'->'ev','v'!='l'->0. f[3]: 'leve'->'eve','ve','e'? 'e'==...no wait compare ends

Actually compare s[0..] with s ending: 'l'=='l' no wait s='level': l,e,v,e,l

f[4]: check if s[0..4] has prefix=suffix. s[0]=l==s[4]=l->f[4]=1? Check s[0..1]='le'==s[3..4]='el'? No.

So f=[0,0,0,0,1]. LPS (Longest Palindromic Subsequence) value = f[-1]=1. Return s[:1]='l' ok

PHASE 5 TEST & DEBUG

WHY KMP failure fn: longest prefix that is also suffix = answer.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
The KMP failure function directly gives the longest proper prefix-suffix."	output or sliding window; O(1) extra if output not counted

Follow-up: LC 214 (Shortest Palindrome) uses the same KMP failure function on s+#+reverse(s) — how does that differ from the failure function on s alone?

IN YOUR APP | Expose | Community